

# PROYECTO INTEGRADOR DE FIN DE ASIGNATURA

## MODERNIZACIÓN E INYECCIÓN CLOUD-NATIVE EN EL SOFTWARE

**Asignatura:**

Computación en la Nube (ISW-341)

**Semestre:**

Séptimo Semestre – Ingeniería de Software

**Institución:**Universidad Católica Boliviana "San Pablo" – Unidad  
Académica Santa Cruz**Docente:**

Ing. Fabrizio Gustavo Bellido Parra

### 1. PRESENTACIÓN DEL DESAFÍO

El proyecto final de la materia Computación en la Nube (ISW-341) busca que cada estudiante demuestre de forma autónoma su capacidad de diseñar, automatizar, desplegar y asegurar aplicaciones utilizando paradigmas modernos de nube, alineándose con las competencias clave detalladas en el "PLAN DE ASIGNATURA COMPUTACION EN LA NUBE".

Para responder a las realidades de la asignatura de forma flexible e individual, el proyecto se divide en dos rutas de implementación diferenciadas. La selección de la ruta define tanto la arquitectura de partida como la plataforma tecnológica (proveedor cloud) obligatoria u opcional que se utilizará para la evaluación.

### 2. RUTAS DE TRABAJO Y ENTORNOS CLOUD

Cada estudiante deberá optar de manera individual por una de las siguientes dos opciones de desarrollo:

#### RUTA A: "Build & Modernize" (Modernización de ABM Propio)

**Paso Inicial (Legacy Monolítico):**

El estudiante diseña y construye desde cero un sistema ABM (Alta, Baja, Modificación – equivalente a CRUD: Create, Read, Update, Delete) completamente funcional sobre una arquitectura monolítica tradicional (ej. Node.js/Express, Python/Django, .NET Core, Java Spring Boot) con una base de datos local y acoplada.

**El Reto:**

Migrar, contenerizar, automatizar e integrar servicios de nube en esta aplicación para transformarla en una solución SaaS escalable.

**Plataforma Cloud Obligatoria: AWS Academy Learner Lab.**

Se debe operar estrictamente bajo las restricciones de esta suscripción:

- Uso obligatorio del rol preconfigurado LabRole e LabInstanceProfile (prohibido crear roles IAM adicionales).
- Control del presupuesto límite de \$50 USD de consumo.
- Autenticación con credenciales temporales que incluyen AWS\_SESSION\_TOKEN (expira cada 4 horas).

#### RUTA B: "Bring Your Own Project" (BYOP – Inyección Cloud en Proyecto Activo)

**Paso Inicial (BYOP – Bring Your Own Project):**

El estudiante utiliza un proyecto de software funcional preexistente o que esté desarrollando activamente en otra asignatura del semestre (ej. Taller de Grado, Desarrollo Web, Dispositivos Móviles, Sistemas de Información).

**El Reto:**

Diseñar e inyectar de manera justificada la infraestructura de nube, pipelines de despliegue y automatización para este proyecto utilizando herramientas modernas.

**Plataforma Cloud: Libre elección (AWS, Microsoft Azure, Google Cloud Platform, u otras).**

Debido a que la arquitectura de cada proyecto activo varía de forma natural, no se limita a AWS Learner Lab. Los estudiantes pueden aprovisionar recursos en las capas gratuitas (Free Tiers) o créditos de cualquier proveedor que se adapte mejor a la naturaleza de su software preexistente, evaluándose con el mismo nivel de exigencia técnica.

### 3. ARQUITECTURAS DE REFERENCIA SEGÚN LA RUTA

#### Arquitectura de Referencia – Ruta A (AWS Learner Lab)

Para la migración de la aplicación ABM, el flujo arquitectónico debe alinearse a las restricciones del laboratorio de AWS:



#### Arquitectura de Referencia – Ruta B (Multi-Cloud / Proveedor Libre)

Para proyectos activos independientes, los estudiantes adaptan la topología a su pila tecnológica (ej. Microsoft Azure o GCP):



### 4. ESTRUCTURA DE LA DOCUMENTACIÓN TÉCNICA

El entregable documental escrito debe presentarse en formato Markdown o PDF, de autoría estrictamente individual, estructurado de la siguiente manera:

1. **Ficha Técnica del Proyecto Base:** Descripción detallada del software inicial (tecnologías, arquitectura de software, modelo de datos y funcionalidad ABM/CRUD).
2. **Definición de Arquitectura de Red y Nube:** Diagrama lógico detallado de la infraestructura final utilizando la simbología oficial del proveedor cloud seleccionado (AWS, Azure o GCP). Debe incluir direccionamiento de red, subredes y flujos de datos.
3. **Registro de Decisiones de Arquitectura (ADR):** Justificación técnica de las soluciones adoptadas (ej. por qué se eligió Azure Web Apps en vez de AKS para reducir la complejidad, o cómo se gestionó la autenticación temporal de AWS Learner Lab).

4. **Guía de Despliegue (Runbook):** Instrucciones claras paso a paso para que el docente pueda replicar, aprovisionar y desplegar el entorno completo utilizando el código generado.
5. **Reporte de Consumo y Costos:** Estimación real del consumo de los recursos de nube aprobados para garantizar la viabilidad económica del sistema, sea para Ruta A, como para la Ruta B.

## 5. RÚBRICA ANALÍTICA DE EVALUACIÓN (100 PUNTOS TOTALES)

La evaluación se compone de una Base Fija Obligatoria (40 Puntos) común para ambas rutas, y un Menú de Pilares Cloud Electivos (60 Puntos) de los cuales el estudiante debe seleccionar e implementar exactamente dos (2) de los cuatro (4) disponibles.

### PARTE I: BASE FIJA OBLIGATORIA (40 PUNTOS)

| Criterio                                                          | Excelente (100%-90%)                                                                                                                                                                                                                                                | Bueno (89%-75%)                                                                                                                                                                                    | En Desarrollo (74%-60%)                                                                                                                                                                        | Pts    |
|-------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| <b>Aplicación Base Funcional (Saber Hacer)</b>                    | La aplicación base (ABM creado en Ruta A o Proyecto Activo de Ruta B) es completamente funcional. El modelo de datos y la interfaz de usuario operan sin errores lógicos ni bugs visibles.                                                                          | La aplicación es funcional en sus flujos principales, pero se detectan pequeños bugs cosméticos o fallas de validación de datos menores.                                                           | La aplicación presenta fallas de persistencia de datos (ej. caídas al guardar/actualizar) o la funcionalidad ABM está incompleta en alguno de sus flujos.                                      | 15 pts |
| <b>Seguridad, Cifrado y Manejo de Secretos (Saber Conocer)</b>    | Los secretos (credenciales de base de datos, API keys, tokens de la nube) están 100% cifrados y desacoplados del código fuente. Se hace uso estricto de secretos de GitHub, Kubernetes Secrets o Parameter Store consumidos de forma segura en tiempo de ejecución. | Se aíslan los secretos de la aplicación en variables de entorno, pero se detectan configuraciones con permisos excesivos en la nube o dependencias desactualizadas con vulnerabilidades conocidas. | Existen malas prácticas menores en el manejo de secretos (ej. archivos .env subidos temporalmente al historial de Git, aunque corregidos posteriormente) o canales de comunicación sin cifrar. | 10 pts |
| <b>Documentación Técnica y Defensa Oral (Saber Conocer / Ser)</b> | Documentación impecable con diagramas arquitectónicos correctos del proveedor, ADRs técnicamente justificadas y Runbook claro. Defensa oral sólida que demuestra un dominio individual e independiente absoluto de los conceptos de computación en la nube.         | Presenta la documentación completa, pero faltan precisiones técnicas en la guía de despliegue o la estimación de costos del consumo de nube es vaga.                                               | Documentación desordenada o incompleta. Durante la defensa individual, el estudiante tiene dificultades conceptuales marcadas para explicar la interacción de los componentes desplegados.     | 15 pts |

**TOTAL BASE OBLIGATORIA: 40 puntos**

## PARTE II: PILARES CLOUD ELECTIVOS (60 PUNTOS)

Cada estudiante debe seleccionar e implementar dos (2) pilares de la siguiente lista según la naturaleza de su proyecto y ruta seleccionada. Cada pilar seleccionado tiene un valor de 30 puntos ( $2 \times 30 = 60$  pts).

Al momento de la entrega, el estudiante deberá declarar formalmente cuáles pilares presentará ante el docente.

### PILAR 1: Contenedores y Orquestación (Docker & Kubernetes)

#### Requisitos:

Dockerización optimizada de la aplicación utilizando técnicas multi-stage y desacoplamiento de la base de datos de la capa de cómputo. La base de datos debe desplegarse como servicio administrado en la nube (RDS, Azure SQL, Cloud SQL) o mediante StatefulSets con PersistentVolumeClaims si se containeriza. Configuración de manifiestos del orquestador (Kubernetes / K3s / AKS / GKE) incluyendo Deployment, Services, ConfigMaps y Secrets, además de políticas de autoescalado (HPA) ante simulación de tráfico.

#### Contexto por Ruta:

- Ruta A (AWS): Se permite desplegar un clúster ligero (K3s o MicroK8s) sobre instancias EC2 utilizando el LabRole para evitar los altos costos y las trabas administrativas de EKS en el Learner Lab.
- Ruta B (Libre): Se evalúa el uso nativo de servicios administrados de Kubernetes (ej. Azure AKS o GCP GKE) o motores de contenedores equivalentes acordes a su arquitectura.

| Nivel de Logro | Descripción del Desempeño                                                                                                                                                                                                                                  | Puntos |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| Excelente      | Imágenes Docker optimizadas (multi-stage builds), uso correcto de Secrets/ConfigMaps en K8s, base de datos desacoplada mediante servicio administrado o PersistentVolumes, despliegue con HPA activo y autoescalado comprobado bajo simulación de tráfico. | 30 pts |
| Bueno          | Contenedores funcionales en el orquestador con manifiestos correctos, pero las variables sensibles están expuestas en texto plano dentro de los archivos YAML, o carece de HPA configurado.                                                                | 24 pts |
| En Desarrollo  | Aplicación containerizada corriendo en el orquestador pero de forma estática (sin mapeo dinámico de servicios, fallas de persistencia de datos o mapeo de puertos incorrecto).                                                                             | 18 pts |
| Insuficiente   | Fallas críticas en las imágenes de Docker (errores continuos de CrashLoopBackOff) o nula configuración del orquestador de contenedores.                                                                                                                    | 9 pts  |

### PILAR 2: Infraestructura como Código (IaC con Terraform)

#### Requisitos:

Aprovisionamiento de la infraestructura de red (VPC, Subredes, Firewalls / Security Groups) y recursos de cómputo/almacenamiento mediante Terraform de manera declarativa. Es obligatorio el uso de módulos reutilizables y el manejo seguro del estado remoto de Terraform (backend en S3/Azure Blob/GCS con bloqueo dinámico cuando sea posible).

#### Contexto por Ruta:

- Ruta A (AWS Learner Lab): El código de Terraform debe configurarse para funcionar con credenciales temporales (aws\_session\_token) y heredar de manera explícita el ARN de LabRole para la creación de recursos. Para el backend remoto, se permite S3 sin DynamoDB lock debido a restricciones del Learner Lab.
- Ruta B (Libre): El código de Terraform debe estar adaptado al proveedor elegido (Azure, GCP, etc.) siguiendo las mejores prácticas de estructura de directorios y modularización nativa.

| Nivel de Logro | Descripción del Desempeño                                                                                                                                                        | Puntos |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| Excelente      | Infraestructura aprovisionada mediante Terraform de forma automatizada, estructurado en módulos reutilizables, parametrizado y con backend remoto funcional (S3/Azure Blob/GCS). | 30 pts |

| Nivel de Logro | Descripción del Desempeño                                                                                                                                    | Puntos |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| Bueno          | Código Terraform funcional pero plano (sin modularización, archivo monolítico) o con persistencia de estado exclusivamente local en el equipo de desarrollo. | 24 pts |
| En Desarrollo  | Terraform compila y despliega recursos sueltos (como servidores virtuales) pero carece de la configuración declarativa de la topología de red completa.      | 18 pts |
| Insuficiente   | Uso nulo o meramente instrumental de Terraform (la gran mayoría de los recursos fueron levantados manualmente desde la consola gráfica del proveedor).       | 9 pts  |

### PILAR 3: Automatización de Pipelines (CI/CD)

#### Requisitos:

Automatización del ciclo de vida del software mediante un pipeline (GitHub Actions, GitLab CI u homólogo). El flujo de ejecución debe activarse por triggers de Git, realizar análisis/pruebas unitarias (mínimo 3 pruebas que validen las operaciones CRUD principales), construir y escanear la seguridad de la imagen de Docker, subirla a un registro privado/público y actualizar el entorno de despliegue en la nube de forma automatizada.

#### Contexto por Ruta:

- Ruta A (AWS): El pipeline debe estar parametrizado para recibir e inyectar dinámicamente el `AWS_SESSION_TOKEN` de AWS Academy como secreto de repositorio para poder autenticar el despliegue automático. Se acepta despliegue semi-automático (build/test/scan automático + deploy manual trigger) debido a la expiración de credenciales cada 4 horas.
- Ruta B (Libre): Se permite el uso de herramientas nativas del proveedor (Azure DevOps, GCP Cloud Build) o herramientas universales parametrizadas para su respectivo flujo de autenticación.

| Nivel de Logro | Descripción del Desempeño                                                                                                                                                                                   | Puntos |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| Excelente      | Pipeline completamente integrado de extremo a extremo (Build, Test con mínimo 3 pruebas unitarias, Security Scan, Push y Deploy automático/semi-automático en la nube) con un manejo impecable de secretos. | 30 pts |
| Bueno          | Pipeline funcional que ejecuta build, test y push de la imagen Docker de forma correcta, pero la actualización final en el entorno de despliegue requiere pasos manuales significativos del estudiante.     | 24 pts |
| En Desarrollo  | El pipeline de automatización se limita a ejecutar pruebas de linter o compilación de software básica, sin conexión o despliegue interactivo con la infraestructura cloud.                                  | 18 pts |
| Insuficiente   | Inexistencia del pipeline de CI/CD. El despliegue de las actualizaciones de software se realiza de manera completamente manual dentro del servidor.                                                         | 9 pts  |

### PILAR 4: Modernización de Servicios y Arquitectura Serverless

#### Requisitos:

Desacoplamiento de tareas asíncronas pesadas (ej. procesamiento de archivos, generación de reportes PDF, envío de notificaciones automáticas o compresión de imágenes) desde la aplicación principal hacia una arquitectura reactiva basada en eventos utilizando almacenamiento de objetos y funciones serverless (FaaS: AWS Lambda, Azure Functions, Cloud Functions).

#### Ejemplos válidos de arquitecturas:

- S3 + Lambda (event-driven file processing)
- API Gateway + Lambda (REST API serverless)

- EventBridge + Lambda (event-driven workflows)
- SQS/SNS + Lambda (message-driven async processing)

**Contexto por Ruta:**

- Ruta A (AWS): Configuración de AWS Lambda asociada a un trigger de S3, EventBridge, SQS o API Gateway, utilizando de manera obligatoria el ARN de LabRole como rol de ejecución de la función.
- Ruta B (Libre): Implementación de arquitecturas sin servidor o basadas en eventos (FaaS / BaaS) compatibles con el proveedor seleccionado y que den soporte lógico al proyecto del estudiante.

| Nivel de Logro | Descripción del Desempeño                                                                                                                                                               | Puntos |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| Excelente      | Integración asíncrona perfecta y reactiva desencadenada por eventos de almacenamiento o mensajes (ej. S3 Trigger → Lambda, SQS → Lambda) comunicada nativamente con la aplicación base. | 30 pts |
| Bueno          | Se implementa la función serverless en la nube, pero esta es invocada de forma síncrona mediante peticiones HTTP directas (sin reaccionar a eventos nativos del proveedor).             | 24 pts |
| En Desarrollo  | La función serverless está desplegada en la nube, pero carece de lógica de integración real con el flujo de datos de la aplicación base o del proyecto del estudiante.                  | 18 pts |
| Insuficiente   | Toda la lógica de procesamiento sigue acoplada dentro del servidor tradicional sin evidencia de evolución hacia arquitecturas Serverless.                                               | 9 pts  |

**MATRIZ DE COMPATIBILIDAD Y RECOMENDACIONES**

Para ayudar en la selección de pilares según la ruta elegida:

| Combinación de Pilares | Ruta A (AWS Learner Lab) | Ruta B (Multi-Cloud) | Observaciones Técnicas                                                                          |
|------------------------|--------------------------|----------------------|-------------------------------------------------------------------------------------------------|
| Pilar 1 + Pilar 2      | ☑ Recomendado            | ☑ Recomendado        | Combinación óptima: K3s/K8s + Terraform. Infraestructura completa automatizada.                 |
| Pilar 1 + Pilar 3      | ⚠ Desafiante             | ☑ Recomendado        | Ruta A requiere estrategia de renovación de credenciales AWS cada 4h. Ruta B sin restricciones. |
| Pilar 1 + Pilar 4      | ☑ Recomendado            | ☑ Recomendado        | K3s/K8s para app principal + Lambda para tareas async. Arquitectura híbrida moderna.            |
| Pilar 2 + Pilar 3      | ⚠ Desafiante             | ☑ Recomendado        | Terraform + CI/CD. Ruta A requiere manejo de tokens temporales en pipeline.                     |
| Pilar 2 + Pilar 4      | ☑ Óptimo                 | ☑ Óptimo             | Infraestructura serverless completa. Bajo costo operativo.                                      |
| Pilar 3 + Pilar 4      | ⚠ Avanzado               | ☑ Recomendado        | CI/CD + Serverless. Ruta A requiere experiencia en manejo de sesiones AWS.                      |

**Leyenda:**

- ☑ Recomendado: Implementación directa y sin restricciones técnicas mayores.
- ⚠ Desafiante/Avanzado: Viable pero requiere estrategias adicionales de configuración.

## 6. REGLA DE SEGURIDAD EXCLUYENTE (FAILSAFE DE SEGURIDAD)

**⚠️ ADVERTENCIA CRÍTICA:** Si en cualquier parte del código fuente desplegado o en el historial del repositorio de Git (incluyendo commits antiguos) se detecta la exposición en texto plano de credenciales de acceso a la nube (AWS\_ACCESS\_KEY\_ID, AWS\_SECRET\_ACCESS\_KEY, AWS\_SESSION\_TOKEN o credenciales de otros proveedores), contraseñas de bases de datos de producción o claves privadas, la nota final asignada al proyecto integrador completo será automáticamente de 0 (cero) puntos, independientemente de la ruta elegida o del nivel de avance técnico del software.

La seguridad y la responsabilidad en el entorno cloud son requisitos indispensables para el ejercicio profesional.

## 7. CÁLCULO DE CALIFICACIÓN FINAL DEL PROYECTO

La calificación final del proyecto integrador se calcula bajo la siguiente relación matemática estricta de suma de puntajes obtenidos:

$$\text{Nota Final} = \text{Parte I (Base Obligatoria)} + \sum_{n=1 \text{ hasta } 2} (\text{Pilar Cloud}_n)$$

Donde:

- Parte I (Base Obligatoria): Suma un máximo de 40 puntos.
- $\sum$  Pilar Cloud\_n: Suma de los dos (2) pilares cloud electivos seleccionados por el estudiante (máximo 30 puntos cada uno = 60 puntos totales).

Ejemplos de Cálculo:

| Escenario                                           | Base Obligatoria  | Pilar 1 | Pilar 2 | Total   |
|-----------------------------------------------------|-------------------|---------|---------|---------|
| Estudiante A (Excelente en todo)                    | 15 + 10 + 15 = 40 | 30      | 30      | 100 pts |
| Estudiante B (Bueno en todo)                        | 13 + 8 + 13 = 34  | 24      | 24      | 82 pts  |
| Estudiante C (Mixto: Base Excelente, Pilares Bueno) | 15 + 10 + 15 = 40 | 24      | 24      | 88 pts  |
| Estudiante D (En Desarrollo)                        | 11 + 6 + 11 = 28  | 18      | 18      | 64 pts  |

## 8. RECURSOS Y ANEXOS TÉCNICOS

### Estrategias para Renovación de Credenciales AWS Learner Lab (Ruta A + Pilar 3)

#### Opción 1: Script de Actualización Manual de Secrets en GitHub

Los estudiantes pueden crear un workflow de GitHub Actions que se ejecute manualmente (workflow\_dispatch) para actualizar las credenciales:

```
name: Update AWS Credentials
on:
  workflow_dispatch:
  inputs:
    aws_access_key:
      description: 'AWS Access Key ID'
      required: true
    aws_secret_key:
      description: 'AWS Secret Access Key'
      required: true
    aws_session_token:
      description: 'AWS Session Token'
      required: true
```

```
jobs:
  update-credentials:
    runs-on: ubuntu-latest
    steps:
      - name: Update Repository Secrets
        run: |
          echo "Credentials updated manually via workflow dispatch"
          # Los secrets se actualizan a través de la UI de GitHub
```

## Opción 2: Pipeline Semi-Automático

Dividir el pipeline en dos workflows:

- Build & Test (Automático): Se ejecuta en cada push y realiza build, tests y security scan.
- Deploy (Manual Trigger): El estudiante actualiza credenciales y ejecuta manualmente el despliegue cuando las necesita.

```
# workflow 1: build-test.yml (automático)
name: Build and Test
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Run Tests
        run: npm test
      - name: Build Docker Image
        run: docker build -t myapp:latest .

# workflow 2: deploy.yml (manual)
name: Deploy to AWS
on:
  workflow_dispatch:
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Configure AWS
        env:
          AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
          AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
          AWS_SESSION_TOKEN: ${ secrets.AWS_SESSION_TOKEN }
        run: |
          aws eks update-kubeconfig --name my-cluster
          kubectl apply -f k8s/
```

## Opción 3: Usar GitHub Container Registry (GHCR)

Construir y almacenar imágenes en GHCR (no requiere credenciales AWS) y solo usar AWS para el despliegue final:

```
- name: Build and Push to GHCR
  run: |
    echo ${ secrets.GITHUB_TOKEN } | docker login ghcr.io -u ${ github.actor } --password-stdin
    docker build -t ghcr.io/${ github.repository }/app:latest .
    docker push ghcr.io/${ github.repository }/app:latest
```

## Template de ADR (Architecture Decision Record)

```
# ADR-001: [Título de la Decisión]

**Fecha:** YYYY-MM-DD
**Estado:** Propuesta | Aceptada | Rechazada | Deprecada
**Decisores:** [Nombre del estudiante]
```



```

## Contexto y Problema
[Descripción del problema técnico o decisión arquitectónica que se debe tomar]

## Opciones Consideradas
1. **Opción A:** [Descripción]
2. **Opción B:** [Descripción]
3. **Opción C:** [Descripción]

## Decisión Tomada
**Opción seleccionada:** [X]

**Justificación:**
- [Razón técnica 1]
- [Razón de costo 2]
- [Razón de compatibilidad con Ruta A/B]

## Consecuencias

**Positivas:**
- [Beneficio 1]
- [Beneficio 2]

**Negativas:**
- [Trade-off 1]
- [Limitación 2]

## Notas Adicionales
[Información complementaria, enlaces a documentación, etc.]

```

### Ejemplo Aplicado:

```

# ADR-001: Uso de K3s en lugar de EKS para Ruta A

**Fecha:** 2026-05-20
**Estado:** Aceptada
**Decisores:** Juan Pérez

## Contexto y Problema
AWS Learner Lab no permite la creación de roles IAM adicionales, y EKS requiere múltiples roles de servicio. Además, el costo estimado de EKS ($0.10/hora por clúster) consumiría $72 del presupuesto de $100 en un mes.

## Opciones Consideradas
1. **EKS (Elastic Kubernetes Service):** Servicio administrado de AWS
2. **K3s en EC2:** Kubernetes ligero autohospedado
3. **ECS (Elastic Container Service):** Alternativa sin Kubernetes

## Decisión Tomada
**Opción seleccionada:** K3s en EC2

**Justificación:**
- No requiere roles IAM adicionales (usa LabInstanceProfile)
- Costo estimado: $15/mes (EC2 t3.medium)
- Cumple requisitos del Pilar 1 (Kubernetes completo)
- Permite aprendizaje de configuración nativa de K8s

## Consecuencias

**Positivas:**
- Control total sobre configuración del clúster
- Bajo costo operativo
- Aprendizaje profundo de Kubernetes

**Negativas:**

```

- Requiere gestión manual del clúster
- Sin alta disponibilidad nativa (single-node)
- Mayor complejidad de configuración inicial

### ## Notas Adicionales

Documentación de K3s: <https://k3s.io/>

Script de instalación incluido en /scripts/install-k3s.sh

## Checklist de Entrega

Antes de la presentación final, verificar que el proyecto incluya:

### Documentación

- Ficha Técnica del Proyecto Base (tecnologías, arquitectura, modelo de datos)
- Diagrama de Arquitectura de Red y Nube (con simbología oficial del proveedor)
- Al menos 2 ADRs documentando decisiones críticas
- Runbook con instrucciones paso a paso de despliegue
- Reporte de Consumo y Costos (con capturas de facturación o estimaciones)

### Código y Repositorio

- Código fuente de la aplicación base (ABM/CRUD funcional)
- Archivos de configuración Docker (Dockerfile, docker-compose.yml si aplica)
- Manifiestos de Kubernetes (si Pilar 1 seleccionado)
- Código de Terraform modularizado (si Pilar 2 seleccionado)
- Configuración de pipeline CI/CD (.github/workflows o .gitlab-ci.yml si Pilar 3)
- Código de función serverless (si Pilar 4 seleccionado)
- Archivo README.md con instrucciones de ejecución local

### Seguridad

- Ningún secreto expuesto en el código o historial de Git
- Archivo .gitignore configurado correctamente (.env, credentials, .terraform, etc.)
- Secrets configurados en GitHub/GitLab o en servicios de la nube
- Variables sensibles inyectadas mediante ConfigMaps/Secrets de K8s o Parameter Store

### Infraestructura Cloud

- Recursos desplegados y funcionales en el proveedor cloud
- Acceso de demostración configurado para el docente (credenciales temporales si es necesario)
- Scripts de aprovisionamiento/destrucción de infraestructura
- Evidencia de monitoreo básico (logs, métricas si aplica)

### Pilares Seleccionados

- Declaración formal de los 2 pilares seleccionados en la documentación
- Evidencia de implementación completa de cada pilar
- Screenshots/videos de demostración de funcionalidades clave

## 9. FECHAS Y MODALIDAD DE ENTREGA

**Fecha límite de entrega:** [Docente definirá la fecha de entrega]

**Modalidad de presentación:**

1. **Entrega de documentación:** Subir a la plataforma LMS del curso en formato PDF.
2. **Repositorio de código:** Compartir URL de repositorio Git en el comentario de la entrega en el LMS (público o con acceso al docente).
3. **Defensa oral individual:** Presentación de 10-15 minutos seguida de preguntas técnicas.

#### Contenido de la defensa oral:

- Demostración en vivo de la aplicación desplegada en la nube
  - Explicación de decisiones arquitectónicas (ADRs)
  - Walkthrough del código de infraestructura (Terraform/K8s manifests/Pipeline)
  - Respuesta a preguntas técnicas del docente sobre implementación
- 

## 10. CRITERIOS DE ORIGINALIDAD Y ÉTICA ACADÉMICA

- El proyecto debe ser de autoría individual estricta. No se permiten trabajos grupales.
  - Está permitido consultar documentación oficial, tutoriales y foros técnicos (Stack Overflow, documentación de AWS/Azure/GCP).
  - No está permitido: Copiar código completo de otros estudiantes, usar proyectos de terceros sin atribución, o presentar trabajo realizado por herramientas de IA sin comprensión conceptual propia.
  - Durante la defensa oral, el docente puede solicitar explicaciones detalladas de cualquier parte del código para verificar comprensión real.
- 

## CONCLUSIÓN

Este proyecto integrador representa la culminación de los conocimientos adquiridos en Computación en la Nube. Se espera que los estudiantes demuestren no solo competencia técnica, sino también capacidad de toma de decisiones arquitectónicas, manejo de restricciones reales de proveedores cloud, y responsabilidad en temas de seguridad.

El éxito en este proyecto es un indicador directo de la preparación del estudiante para enfrentar desafíos profesionales en el ámbito de DevOps, Cloud Engineering y desarrollo de aplicaciones nativas de la nube.

**¡Mucho éxito en el desarrollo de su proyecto!**

---

### Documentación de Referencia

AWS Academy Learner Lab: <https://awsacademy.instructure.com/>

Terraform Documentation: <https://www.terraform.io/docs>

Kubernetes Documentation: <https://kubernetes.io/docs/>

GitHub Actions: <https://docs.github.com/en/actions>

AWS Lambda: <https://docs.aws.amazon.com/lambda/>

Azure Functions: <https://docs.microsoft.com/en-us/azure/azure-functions/>

Google Cloud Functions: <https://cloud.google.com/functions/docs>

LMS: <https://lms.ucb.edu.bo/course/view.php?id=18503>